

Reviewer Pack — Minimal Rank of tropical power series over $\mathbb{R}$ vs Communication Complexity...

Entry af2f5733f52c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 06:35:23 UTC

Minimal Rank of tropical power series over $\mathbb{R}$ vs Communication Complexity for Disjointness

Entry ID: af2f5733f52c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 06:35:23 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity

Statement:

[‘For any $|X|$ -by- $|Y|$ matrix M , the minimum rank of its tropicalization $\tau(M)$ over the real numbers, $\tau_{\mathbb{R}}(M)$, is a lower bound on the randomized communication complexity of the partial function defined by M , such that for the disjointness problem, $\tau_{\mathbb{R}}(\text{DISJ}_n) = \Omega(n)$.’, ‘Equivalently, there exists an absolute constant $c > 0$ such that for all Boolean matrices A representing a disjointness instance over n variables, the rank of its tropicalization A_{trop} satisfies: $\text{rank}(A_{\text{trop}}) \geq c \cdot n$.’, ‘Further, this lower bound is independent of the specific choice of representation of the partial function.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a framework for studying problems in combinatorics and complexity theory. The tropicalization of matrices has been used to analyze communication complexity, but its potential as a lower bound for disjointness remains unexplored.’, ‘This conjecture suggests that the structure captured by tropical rank could offer new insights into the fundamental limitations of communication complexity.’, ‘If true, it would provide a non-trivial connection between tropical geometry and communication complexity.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8c7fbd9740ae4046

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The tropical rank of matrices’ tropicalization meets the lower bound on communication complexity for disjointness, specifically if the mean rank across all seeds is at least $c \cdot n$ where c is an absolute constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "communication complexity" AND "disjointness problem" - "minimal rank tropical power series" AND $\tau_R(M)$ AND communication complexity" - "absolute constant $c > 0$ " AND $\text{rank}(A_{\text{trop}}) \geq c \cdot n$ AND disjointness problem

Top relevant hits considered: - [<http://arxiv.org/abs/2404.00641v2>] Polynomial Bogolyubov for special linear groups via tensor rank - [<http://arxiv.org/abs/2510.12933v1>] On random matrices with large corank - [<http://arxiv.org/abs/2310.19136v2>] Outlier-robust additive matrix decomposition

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tropicalize(matrix):
        return [[max(row[j] for row in matrix) for j in range(len(matrix[0]))] for i in range(len(matrix))]

    def rank(matrix):
        m, n = len(matrix), len(matrix[0])
        if m == 0 or n == 0:
            return 0
        matrix = [row[:] for row in matrix]
        lead = 0
        while lead < n and all(matrix[i][lead] == -math.inf for i in range(m)):
            lead += 1
        if lead == n:
            return 0
        for i in range(m):
            if matrix[i][lead] != -math.inf:
                for j in range(n):
                    if j != lead:
                        matrix[i][j] -= matrix[i][lead]
                matrix[i][lead] = -matrix[i][lead]
        for i in range(m):
```

```

        if i != 0 and matrix[i][lead] != -math.inf:
            factor = matrix[i][lead] / matrix[0][lead]
            for j in range(n):
                matrix[i][j] -= factor * matrix[0][j]
    return 1 + rank([matrix[i][j] for j in range(lead, n)] for i in range(1, m) if any(matrix[i][j] != -math.inf))

def generate_disjointness_matrix(n):
    A = [[0 if i == j else 1 for j in range(n)] for i in range(n)]
    return A

n_values = [5, 10, 15, 20, 30, 40]
total_rank = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Ensure at least 5 instances per size
        A = generate_disjointness_matrix(n)
        A_trop = tropicalize(A)
        rank_A_trop = rank(A_trop)
        total_rank += rank_A_trop
        instances_tested += 1

mean_rank = total_rank / instances_tested
conjecture_holds = mean_rank >= n_values[0] * 0.5 # Arbitrary constant c > 0, here we use 0.5
counterexample = "" if conjecture_holds else f"Mean rank {mean_rank} is less than {n_values[0]}"

return {
    "metric_name": "Tropical Rank",
    "metric_value": mean_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)][\"counterexample\"]}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02ca1093.py", line 81, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02ca1093.py", line 59, in run_trial
    rank_A_trop = rank(A_trop)
                ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02ca1093.py", line 45, in rank
    return 1 + rank([[matrix[i][j] for j in range(lead, n)] for i in range(1, m) if any(matrix[i][j] != -
math.inf for j in range(lead, n))])
                ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02ca1093.py", line 42, in rank
    factor = matrix[i][lead] / matrix[0][lead]
                ~~~~~^~~~~~
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes without crashing and verify if the support condition is met.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12367
2	preregistration	ollama_remote	glm4:latest	0	0	5375
3	novelty	ollama_remote	glm4:latest	0	0	4776
4	novelty	ollama_remote	glm4:latest	0	0	5372
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12375
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9801
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10305
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11451
9	judge	ollama_remote	glm4:latest	0	0	8166

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 79988 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/af2f5733f52c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/af2f5733f52c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/af2f5733f52c.tar.gz (if generated)